CHAITANYA BHARATHI INSTITUTE OF TECHNOLOGY
An Autonomous Institute | Affiliated to Osmania University
Kokapet Village, Gandipet Mandal, Hyderabad, Telangana-500075, www.cbit.ac.in
Approved by | Affiliated to | UGC Autonomous | 14 Programs Accredited by NBA | Grade A++ in NAAC | All India Ranking 151-200 Band nirf | Certifications Quality ISO 21001 : 2018 Green ISO 14001 : 2015 Energy ISO 50001 : 2018
COMMITTED TO RESEARCH, INNOVATION AND EDUCATION 48 years

AI-POWERED E-COMMERCE INTELLIGENCE PLATFORM

CHAITANYA BHARATHI INSTITUTE OF TECHNOLOGY (A)

(Affiliated to Osmania University)

CASE STUDY REPORT

Submitted By:

Akshay – 160124748113
Raja Sanjay – 160124748110

CASE STUDY GUIDE

Panigrahi Srikanth, Assistant Professor

Department of CSE Artificial Intelligence and Machine Learning
CBIT, Hyderabad

Certificate

This is to certify that the case study report entitled **"AI-Powered E-Commerce Intelligence Platform"** has been prepared by the following students during the academic year 2025–2026 under the guidance and supervision of the undersigned.

Akshay – 160124748113
Raja Sanjay – 160124748110

Project Guide:

Panigrahi Srikanth
Assistant Professor
Department of CSE Artificial Intelligence and Machine Learning
CBIT, Hyderabad

Acknowledgement

We express our sincere gratitude to our case study guide, **Panigrahi Srikanth**, Assistant Professor, for his guidance, encouragement, and valuable suggestions throughout this work. His feedback helped us organize the experiments and present the work in a systematic manner.

We thank the Department of Computer Science and Engineering, Artificial Intelligence and Machine Learning, Chaitanya Bharathi Institute of Technology, for providing the academic environment and resources required to complete this case study.

We also acknowledge the contributors of the Fashion Product Images (Small) dataset and the developers of the open-source Python libraries used for data processing, visualization, and model development. Finally, we thank our family and friends for their continued support and motivation.

Abstract

Electronic commerce platforms manage large product catalogues containing product photographs and descriptive attributes. Organizing these catalogues manually can be time-consuming and inconsistent. This case study investigates an AI-powered workflow for analysing fashion product data using metadata-based machine learning and image-based deep learning.

The experiments use the Fashion Product Images (Small) dataset, including the metadata file `styles.csv` and corresponding product images. The workflow covers data preparation, perceptron-based classification, product-category classification from categorical metadata, product-year estimation, optimizer comparison, regularization experiments, a convolutional neural network trained from scratch, and transfer learning with VGG16 and ResNet50.

The notebook visualizes training behaviour, optimizer performance, regularization experiments, a confusion matrix, and model accuracy comparisons. In the saved run, the perceptron image experiment reports an accuracy of 0.9700. The same-test-set image-model comparison reports approximately 0.719 accuracy for the CNN trained from scratch, 0.908 for VGG16, and 0.908 for ResNet50. These values describe the saved experiment and may change if the notebook is rerun with different software or execution conditions.

The case study demonstrates the complementary roles of product metadata and visual features and shows why model selection should be based on measured performance rather than architecture alone.

List of Figures

4.1 Perceptron training error for topwear-versus-bottomwear image classification. . 10
4.2 Metadata experiment results: category-classification accuracy and predicted versus actual product year. 10
4.3 Training loss and final test accuracy for the four optimizer configurations. . . . 11
4.4 Training and validation accuracy curves for baseline and regularized models. . . 12
4.5 Validation accuracy for the tested L2 regularization and dropout configurations. 12
4.6 Training and validation accuracy and loss for the CNN trained from scratch. . . 13
4.7 Confusion matrix for the CNN product-image classifier. 13
4.8 Validation accuracy and validation loss for VGG16 and ResNet50 transfer learning. 14
4.9 Same-test-set accuracy comparison for the CNN trained from scratch, VGG16, and ResNet50. 14

List of Tables

3.1 Hardware requirements for the experiments 6
3.2 Software tools and their roles 7

5.1 Main input and experiment settings 16

6.1 Image-model test accuracy in the saved notebook run 18

Table of Contents

Certificate i

Acknowledgement ii

Abstract iii

List of Figures iv

List of Tables v

1 Introduction 1

1.1 Background and Context 1
1.2 Problem Statement 1
1.3 Motivation 1
1.4 Objectives 2
1.5 Scope of the Study 2
1.6 Organization of the Report 2

2 Literature Review and Theoretical Study 3

2.1 Artificial Intelligence for Product Catalogue Analysis 3
2.2 Perceptron Model 3
2.3 Multilayer Perceptron 4
2.4 Regression for Product-Year Estimation 4
2.5 Convolutional Neural Networks 4
2.6 Transfer Learning: VGG16 and ResNet50 4
2.7 Optimization Algorithms 4
2.8 Regularization and Generalization 5

3 System Requirements and Specifications 6

3.1 Hardware Requirements 6
3.2 Software and Frameworks 6
3.3 Dataset Description and Characteristics 7

3.4 Data Preparation 7
3.5 Evaluation Metrics 8

4 Methodology and System Design **9**
4.1 Overall Methodology Workflow 9
4.2 Perceptron Experiment 9
4.3 Metadata-Based Category Classification 10
4.4 Product-Year Estimation 10
4.5 Optimizer Comparison 11
4.6 Regularization Experiments 11
4.7 CNN Trained from Scratch 12
4.8 Transfer Learning with VGG16 and ResNet50 13

5 Implementation Details and Experimental Setup **15**
5.1 Implementation Environment 15
5.2 Dataset Filtering and Class Selection 15
5.3 Input Processing 15
5.4 Model Training 16
5.5 Reproducibility Considerations 16

6 Results and Performance Analysis **17**
6.1 Overview 17
6.2 Perceptron Performance 17
6.3 Metadata Classification and Regression 17
6.4 Optimizer Comparison 18
6.5 Regularization and Overfitting 18
6.6 CNN Training and Error Analysis 18
6.7 Comparison of Image Models 18
6.8 Overall Discussion 19

7 Conclusion and Future Scope **20**
7.1 Conclusion 20
7.2 Limitations 20
7.3 Future Scope 21

References **22**

Introduction

1.1 Background and Context

Electronic commerce has expanded the volume and variety of products available to customers. A typical catalogue can contain thousands of items, each represented by an image and structured attributes such as gender, colour, season, usage, master category, subcategory, and article type. Consistent categorization is important for search, filtering, recommendations, and catalogue maintenance.

Manual product tagging can require considerable effort and may introduce inconsistent labels. Machine learning offers a way to learn patterns from existing product records and apply those patterns to additional records. Metadata-based models use the structured attributes already stored in a catalogue, while image-based models learn visual representations directly from product photographs.

1.2 Problem Statement

This case study investigates how artificial intelligence techniques can support automated fashion-product analysis. The principal tasks are to classify product categories from metadata, estimate product year from available attributes, and classify product images. The study also compares optimizers and regularization approaches and evaluates whether transfer learning improves image classification compared with a convolutional neural network trained from scratch.

1.3 Motivation

A product catalogue is useful only when its records can be searched and grouped reliably. Automated classification can assist catalogue teams by suggesting categories, identifying inconsis-

tent labels, and reducing repetitive manual work. Image-based models can provide additional evidence when metadata is incomplete or insufficiently descriptive. These potential benefits motivate the comparison of structured-data and image-based approaches.

1.4 Objectives

The objectives of the case study are:

1. Load and prepare product metadata and the corresponding image files.
2. Demonstrate perceptron learning through logical AND and OR operations.
3. Apply a perceptron to binary topwear-versus-bottomwear image classification.
4. Train a multilayer perceptron to classify frequent article types from categorical metadata.
5. Explore product-year estimation as a regression task.
6. Compare Stochastic Gradient Descent, Momentum, RMSProp, and Adam.
7. Study overfitting and compare baseline and regularized models.
8. Train a convolutional neural network (CNN) from scratch.
9. Compare the CNN with transfer-learning models based on VGG16 and ResNet50.

1.5 Scope of the Study

The work is an experimental notebook-based case study, not a deployed commercial service. The experiments use selected classes and a capped image subset to keep training manageable. The findings are specific to the dataset, preprocessing steps, training configuration, and saved test split. They should not be interpreted as a guarantee of performance on every fashion catalogue or e-commerce platform.

1.6 Organization of the Report

Chapter 1 introduces the problem and objectives. Chapter 2 discusses the theoretical concepts behind the models. Chapter 3 describes the dataset, system requirements, and data preparation. Chapter 4 explains the methodology and model architectures. Chapter 5 documents implementation and experimental settings. Chapter 6 discusses the observed results and model comparisons. Chapter 7 summarizes conclusions, limitations, and future work.

Literature Review and Theoretical Study

2.1 Artificial Intelligence for Product Catalogue Analysis

Machine learning can support product classification, attribute prediction, recommendation, and catalogue quality control. Structured attributes are often convenient for baseline models because they are compact and relatively easy to process. Product images provide richer visual information, but they generally require more computation and careful preprocessing. A combined system could use both sources of information, although this notebook evaluates the approaches primarily as separate experiments.

2.2 Perceptron Model

A perceptron is a simple artificial neuron used for binary classification. It calculates a weighted sum of the input features and a bias term:

$$z = \sum_{i=1}^{n} w_i x_i + b.$$

The predicted class is given by the threshold rule:

$$\hat{y} = \begin{cases} 1, & z \geq 0, \\ 0, & z < 0. \end{cases}$$

The learning rule updates the weights according to the difference between the target and predicted label:

$$w_i \leftarrow w_i + \eta(y - \hat{y})x_i, \qquad b \leftarrow b + \eta(y - \hat{y}),$$

where η is the learning rate. A perceptron is interpretable and computationally simple, but a single linear decision boundary cannot represent every possible relationship in complex data.

2.3 Multilayer Perceptron

A multilayer perceptron contains an input layer, one or more hidden layers, and an output layer. Nonlinear activation functions allow hidden layers to learn more complex relationships than a single perceptron. In this case study, categorical product attributes are encoded numerically before being supplied to a dense neural network. A softmax output layer produces class probabilities for article-type classification.

2.4 Regression for Product-Year Estimation

Classification predicts a discrete class, whereas regression predicts a numerical value. Product year is treated as a numerical target in the notebook's year-estimation experiment. The model learns a mapping from encoded product attributes to a year estimate. Regression performance should be interpreted using an appropriate numerical error metric, and the scatter plot of predicted versus actual values helps reveal whether predictions track the target range.

2.5 Convolutional Neural Networks

Convolutional neural networks are designed to process grid-like data such as images. Convolutional filters learn local patterns, including edges, textures, and shapes. Pooling layers reduce spatial dimensions and computation, while later layers combine local patterns into higher-level representations. The notebook's CNN uses convolutional layers, max pooling, global average pooling, a dense layer, dropout, and a softmax output layer.

2.6 Transfer Learning: VGG16 and ResNet50

Transfer learning reuses representations learned by a model trained on a large dataset. VGG16 uses a sequence of convolutional layers, while ResNet50 uses residual connections to support the training of deeper networks. In this project, ImageNet-pretrained bases are frozen and a new classification head is trained for the selected product categories. Freezing the base reduces the number of trainable parameters and the computational burden, but it also means the base features are not adapted through fine-tuning to the target data.

2.7 Optimization Algorithms

The optimizer determines how model parameters are updated during training. Stochastic Gradient Descent (SGD) uses gradient information to update parameters. Momentum accumulates

part of the previous update direction. RMSProp adapts updates using a moving average of squared gradients. Adam combines adaptive learning rates with estimates of first and second gradient moments. Their behaviour depends on the learning rate, data, architecture, and number of epochs; no optimizer is guaranteed to be best for every task.

2.8 Regularization and Generalization

Overfitting occurs when a model learns patterns specific to the training set that do not generalize well to unseen data. L2 regularization penalizes large weights, dropout randomly disables activations during training, and early stopping halts training when validation loss stops improving. Training and validation curves help diagnose whether a model is overfitting or underfitting. These techniques do not guarantee higher test accuracy, so their effect must be measured.

System Requirements and Specifications

3.1 Hardware Requirements

The experiments can run in a Python notebook environment. A modern multi-core processor and adequate memory are needed for metadata processing. A graphics processing unit is recommended for faster CNN and transfer-learning experiments, although exact requirements depend on image count, batch size, and available memory.

Table 3.1: Hardware requirements for the experiments

Component	**Purpose / requirement**
Processor	Modern multi-core central processing unit for data preparation and general execution.
Memory	Sufficient random-access memory to load metadata and batches of images.
Storage	Space for the dataset, notebook, saved plots, and model outputs.
Graphics processor	Recommended for reducing image-model training time; not essential for all metadata experiments.
Internet access	Required to download the dataset and, where needed, pretrained model weights.

3.2 Software and Frameworks

The notebook uses Python and standard data-science and deep-learning libraries. The exact versions may depend on the execution environment.

Table 3.2: Software tools and their roles

Tool / library	**Role in the project**
Python	Main programming language for data processing and modelling.
NumPy	Numerical arrays and mathematical operations.
Pandas	Loading, cleaning, and filtering product metadata.
TensorFlow / Keras	Defining, training, and evaluating neural networks.
Scikit-learn	Data splitting, encoding, preprocessing, and evaluation utilities.
Matplotlib and Seaborn	Plotting training curves, comparisons, and confusion matrices.
Kaggle API	Downloading the Fashion Product Images (Small) dataset.

3.3 Dataset Description and Characteristics

The notebook uses the Fashion Product Images (Small) dataset by Param Aggarwal. The metadata file `styles.csv` contains descriptive attributes for fashion products, and image files are associated with product identifiers. Fields used or examined in the preparation workflow include master category, subcategory, article type, base colour, season, usage, year, and gender.

The notebook removes records with missing required values and verifies that the expected image file exists. For the principal multiclass experiments, the ten most frequent article types are selected. Image experiments use a capped subset per class to control runtime and memory requirements. As a result, the experiments represent a selected portion of the full dataset rather than every available product.

3.4 Data Preparation

The preparation workflow is summarized below:

1. Download and extract the dataset.
2. Load the product metadata from `styles.csv`.
3. Remove records with missing values in the fields required by the experiment.
4. Construct image paths using the product identifier and check that the files exist.
5. Select the frequent article types required for multiclass classification.
6. Encode categorical fields for use by neural network models.
7. Create training, validation, and test subsets according to each experiment's setup.

8. Resize images and apply the model-specific preprocessing required by the CNN or pre-trained network.

3.5 Evaluation Metrics

Accuracy is the fraction of predictions that are correct:

$$\text{Accuracy} = \frac{\text{Number of correct predictions}}{\text{Total predictions}}.$$

Precision measures the proportion of predicted positives that are correct, while recall measures the proportion of actual positives that are identified. The F1-score is the harmonic mean of precision and recall:

$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN},$$

$$F_1 = 2\frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}.$$

Here, TP, FP, and FN denote true positives, false positives, and false negatives. A confusion matrix presents the distribution of true and predicted classes and helps identify categories that are commonly confused.

Methodology and System Design

4.1 Overall Methodology Workflow

The case study is organized into sequential experiments. Data preparation is performed first so that metadata and image files can be used consistently. Simple perceptron experiments provide a baseline, after which dense neural networks are used for metadata tasks. Optimizer and regularization experiments investigate training behaviour. Finally, image classification is performed using a CNN trained from scratch and transfer-learning models.

1. Dataset download, inspection, and cleaning.
2. Perceptron demonstration for logical AND and OR.
3. Binary image classification for topwear and bottomwear.
4. Multiclass classification from categorical product metadata.
5. Product-year regression from metadata.
6. Optimizer comparison and regularization experiments.
7. CNN image classification and confusion-matrix analysis.
8. Transfer-learning comparison using VGG16 and ResNet50.

4.2 Perceptron Experiment

The perceptron section first demonstrates simple logical operations. It then applies a perceptron to a binary image task that distinguishes topwear from bottomwear. The training-error plot records the number of misclassifications across epochs. The decrease in error over the recorded epochs indicates that the model learns useful separating information, although some classification errors remain.

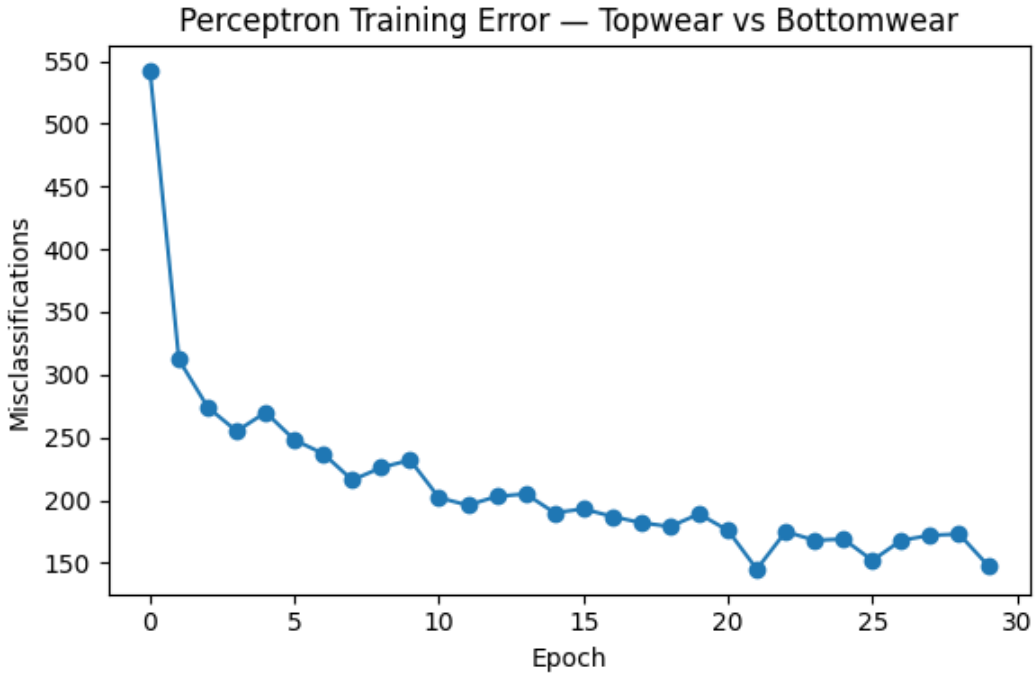


Figure 4.1: Perceptron training error for topwear-versus-bottomwear image classification.

4.3 Metadata-Based Category Classification

Gender, base colour, season, and usage are categorical inputs in the metadata classification experiment. The notebook encodes these fields and trains a multilayer perceptron to predict one of the ten selected article types. The dense network uses hidden layers and a softmax output layer, with sparse categorical cross-entropy as the classification loss. The plotted results include category-classification accuracy and a product-year regression visualization.

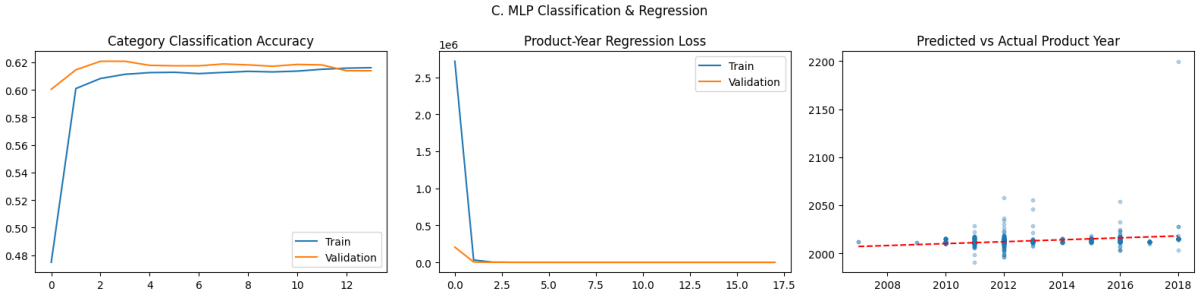


Figure 4.2: Metadata experiment results: category-classification accuracy and predicted versus actual product year.

4.4 Product-Year Estimation

The product-year model treats year as a numerical target. It learns from encoded metadata attributes and generates a year estimate for each input. The predicted-versus-actual plot is

useful for checking whether predictions follow the target range, and the training and validation loss curves indicate how the regression model behaves during optimization. Year prediction should be interpreted as an experimental task because the metadata fields may not contain enough information to determine a product's year precisely.

4.5 Optimizer Comparison

The notebook compares SGD, Momentum, RMSProp, and Adam using the configured dense model and training setup. Loss curves show how quickly the optimization objective decreases, while the final-accuracy chart compares classification performance on the evaluated data. For a fair comparison, model architecture and data split should remain consistent while the optimizer configuration changes.

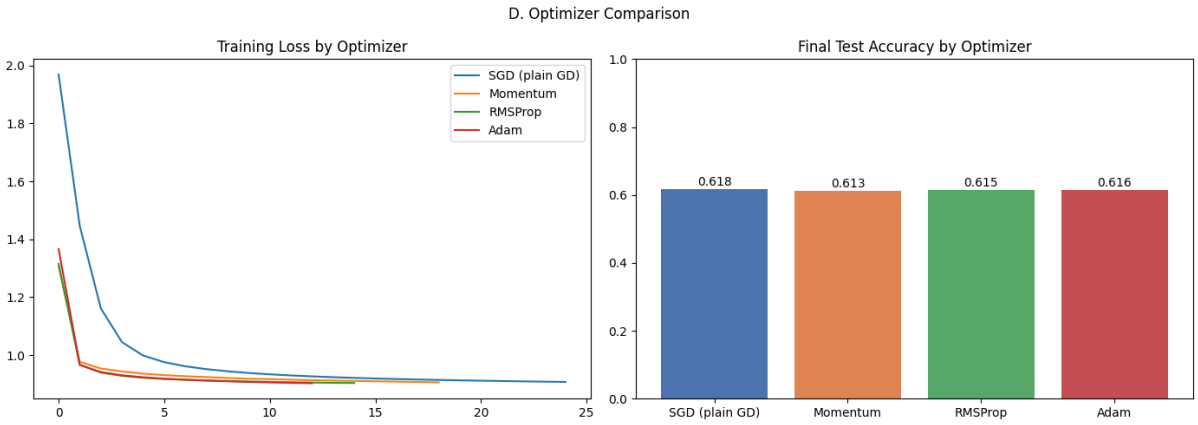


Figure 4.3: Training loss and final test accuracy for the four optimizer configurations.

4.6 Regularization Experiments

The regularization experiment compares a baseline model with a model using L2 weight regularization, dropout, and early stopping. Training and validation curves show the extent to which the model fits the training data relative to unseen validation examples. The notebook also tests combinations of L2 strength and dropout rate, plotting validation accuracy for the tested configurations.

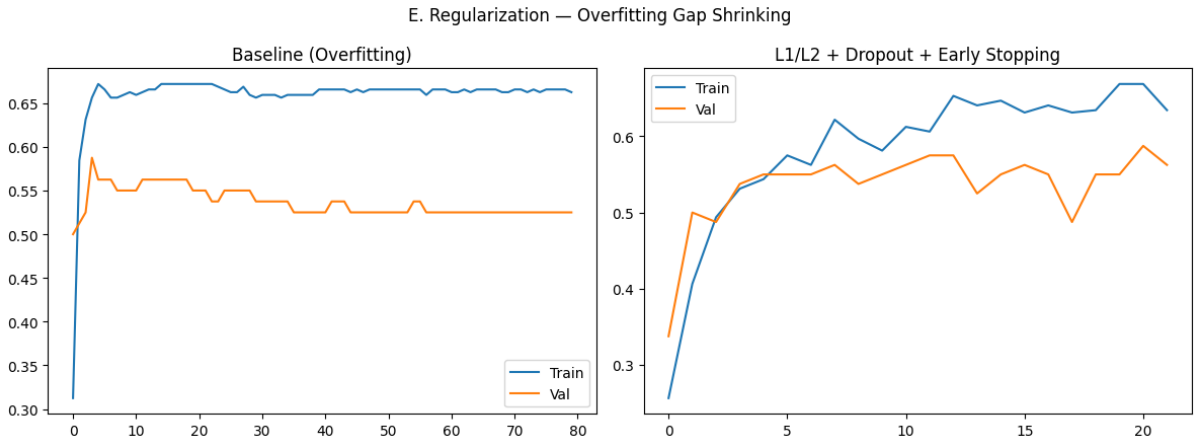


Figure 4.4: Training and validation accuracy curves for baseline and regularized models.

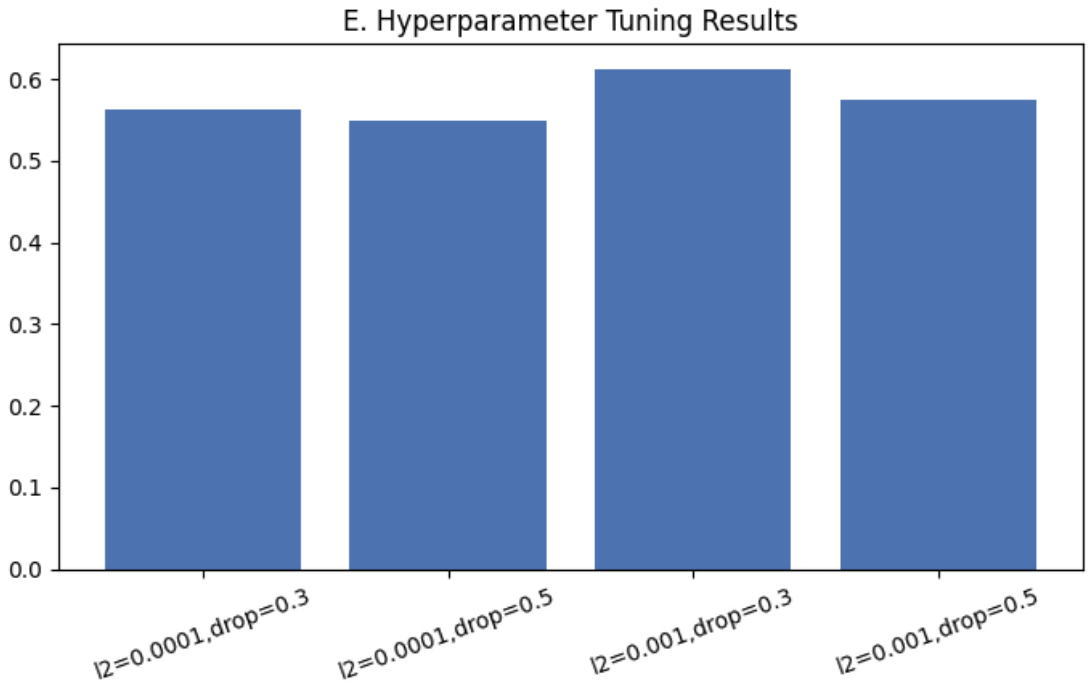


Figure 4.5: Validation accuracy for the tested L2 regularization and dropout configurations.

4.7 CNN Trained from Scratch

The image classifier trained from scratch uses product images resized to 96 by 96 pixels. Its architecture consists of convolutional layers, max-pooling layers, global average pooling, a dense layer, dropout, and a softmax output layer. Training and validation accuracy and loss are tracked over epochs. A confusion matrix is then used to inspect the class-wise prediction patterns on the test set.

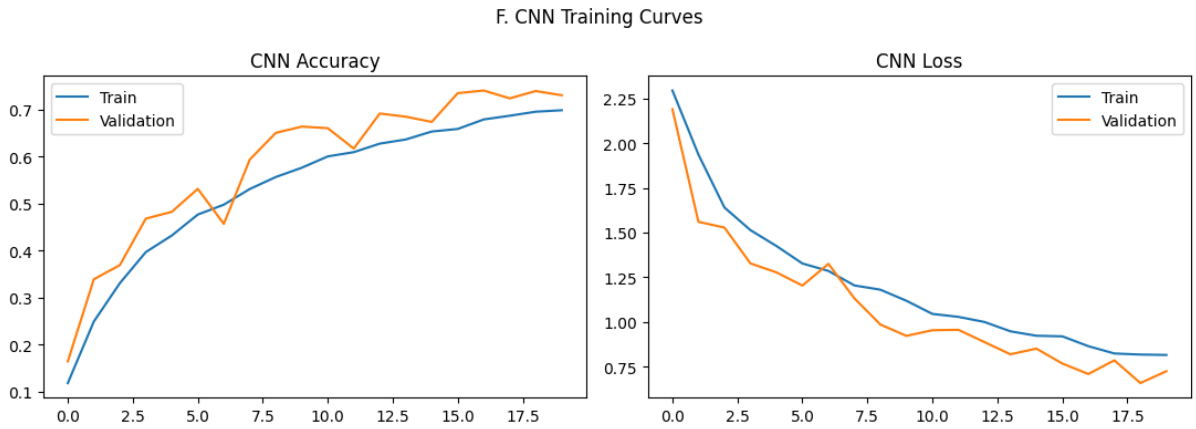


Figure 4.6: Training and validation accuracy and loss for the CNN trained from scratch.

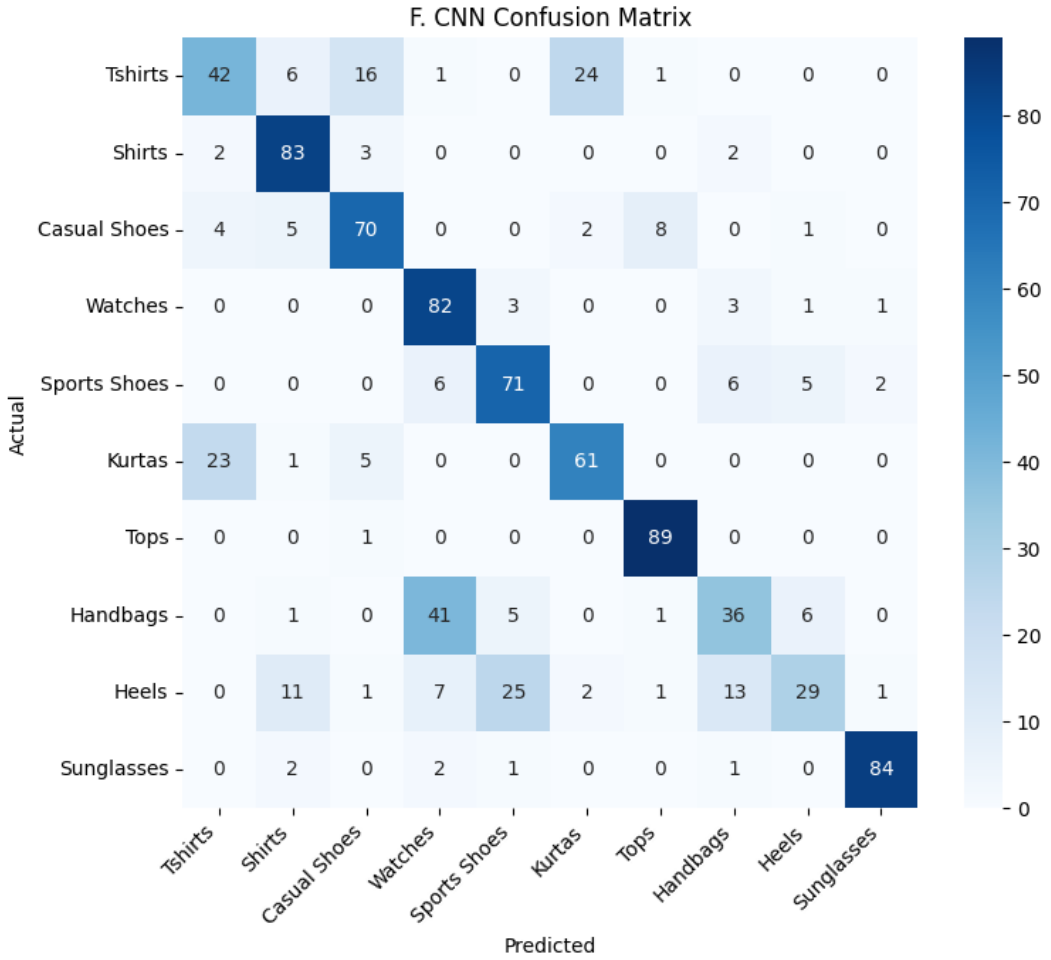


Figure 4.7: Confusion matrix for the CNN product-image classifier.

4.8 Transfer Learning with VGG16 and ResNet50

The transfer-learning experiments use VGG16 and ResNet50 pretrained on ImageNet. The pre-trained convolutional bases are frozen, and a new classification head is trained for the selected

product categories. Images are resized to 128 by 128 pixels and processed using the relevant model input pipeline. The learning curves compare validation accuracy and loss for the two architectures.

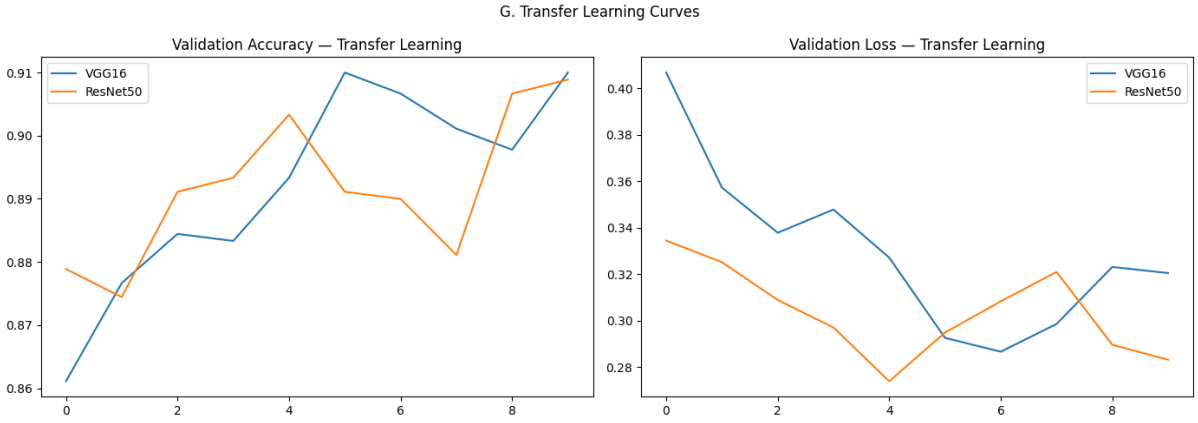


Figure 4.8: Validation accuracy and validation loss for VGG16 and ResNet50 transfer learning.

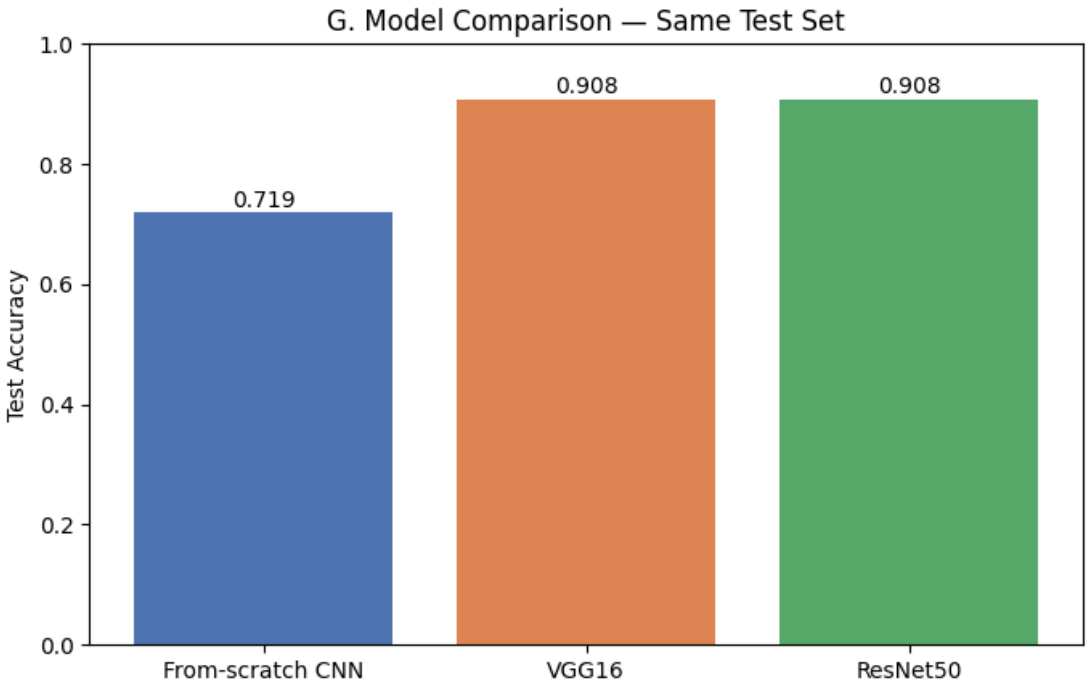


Figure 4.9: Same-test-set accuracy comparison for the CNN trained from scratch, VGG16, and ResNet50.

Implementation Details and Experimental Setup

5.1 Implementation Environment

The experiments are implemented in a Python notebook. Pandas is used to inspect and filter metadata, NumPy supports numerical operations, and TensorFlow/Keras defines and trains the neural networks. Scikit-learn supports preprocessing and data splitting, while Matplotlib and Seaborn are used for plotting. Dataset download relies on the Kaggle API and requires an appropriately configured environment.

5.2 Dataset Filtering and Class Selection

The notebook first cleans the metadata and checks the associated image paths. It selects the ten most frequent article types for the principal multiclass task. Image-heavy experiments cap the number of images per class to make training more manageable. This design reduces the computational burden, but the resulting class distribution and subset selection should be considered when interpreting results.

5.3 Input Processing

Categorical metadata is converted to numeric form before being passed to dense networks. Image models use resized RGB images and model-specific preprocessing. The perceptron image experiment uses smaller images to simplify the binary classification task, while the CNN and transfer-learning models use larger inputs to retain more visual detail.

Table 5.1: Main input and experiment settings

Setting	Description from the notebook workflow
Random seed	42 is set for NumPy and TensorFlow to improve reproducibility.
Target classes	Ten most frequent article types for the main multiclass experiments.
Image subset	A cap of up to 600 images per class is used for image experiments.
Perceptron image input	Images are reduced to 28-pixel dimensions for the binary task.
CNN image input	Images are resized to 96 by 96 pixels.
Transfer-learning input	Images are resized to 128 by 128 pixels.
Training controls	Validation data, learning curves, and early stopping are used where configured.

5.4 Model Training

The metadata classifier is trained with a multiclass classification loss. The product-year model is trained as a regression problem. Optimizer comparison keeps the general task and model structure consistent while changing the optimization method. Regularization experiments vary weight penalties and dropout, with early stopping monitoring validation loss where configured.

For image classification, the CNN learns its feature representation from the selected training images. The transfer-learning models use fixed pretrained convolutional bases and train a new classification head. All reported comparisons should use the designated test set and should not use test performance to tune model parameters.

5.5 Reproducibility Considerations

The notebook sets random seeds and defines the major model configurations, but exact results may still vary because of software versions, hardware kernels, nondeterministic operations, or changes in dataset availability. The numerical values in this report refer to the saved notebook run. If the notebook is rerun, its newly generated outputs should be checked before replacing the reported values.

Results and Performance Analysis

6.1 Overview

The experiments evaluate both metadata-driven and image-driven approaches. The perceptron provides a simple baseline for binary classification. The metadata network evaluates whether structured product attributes can predict article types, while the CNN and transfer-learning models use image content. The notebook's saved plots are reproduced in the methodology chapter and referenced below for interpretation.

6.2 Perceptron Performance

The saved notebook reports an accuracy of 0.9700 for the topwear-versus-bottomwear perceptron experiment. Figure 4.1 shows misclassifications decreasing substantially during the early epochs and then fluctuating at a lower level. The trend suggests that the model learns a useful decision boundary. The remaining errors indicate that the selected image representation and linear classifier do not perfectly separate the two groups.

6.3 Metadata Classification and Regression

Figure 4.2 presents the metadata classifier's accuracy history and the product-year regression visualization. The category classifier learns from gender, colour, season, and usage attributes. These features provide useful signals but cannot fully represent every visual or contextual difference between article types. The regression plot should be examined for the spread of predicted values around actual years; a narrow range of predictions may indicate that the model is not capturing the full variation in the target.

6.4 Optimizer Comparison

Figure 4.3 compares the loss curves and final accuracy of SGD, Momentum, RMSProp, and Adam. The plotted curves show that the optimizers reduce loss at different rates. The final accuracy bars are close to one another in the saved experiment, so differences should be considered together with the training curves and repeated-run variability. A single run is not sufficient to establish that one optimizer is universally superior.

6.5 Regularization and Overfitting

Figure 4.4 compares baseline and regularized training behaviour. The baseline plot exhibits a gap between training and validation performance, which is consistent with overfitting. The regularized model changes the learning trajectory through L2 regularization, dropout, and early stopping. Figure 4.5 shows validation accuracy for the tested combinations of L2 strength and dropout rate. The best configuration should be chosen using validation performance, not the test set.

6.6 CNN Training and Error Analysis

Figure 4.6 shows that training and validation accuracy generally increase as loss decreases. The curves help show whether learning continues to improve and whether validation behaviour follows training behaviour. Figure 4.7 provides a class-level view of errors. Some categories are more easily confused than others, which may reflect similar visual appearance, variation within a category, or limited examples in the selected subset.

6.7 Comparison of Image Models

The saved same-test-set comparison reports the following approximate accuracies:

Table 6.1: Image-model test accuracy in the saved notebook run

Model	**Test accuracy**
CNN trained from scratch	0.719 (71.9%)
VGG16 transfer learning	0.908 (90.8%)
ResNet50 transfer learning	0.908 (90.8%)

Figure 4.9 visualizes the same comparison. In the saved run, both transfer-learning models outperform the CNN trained from scratch. This result is consistent with the benefit of reusing

visual representations learned from a larger image collection. VGG16 and ResNet50 have the same displayed test accuracy in this experiment, so other considerations such as inference time, model size, and repeatability would be needed to choose between them.

6.8 Overall Discussion

The experiments show that model performance depends on the type of input, architecture, training strategy, and selected data subset. Metadata-based models are relatively lightweight and can use attributes already stored in the catalogue. Image-based models can learn visual signals that are not explicitly represented in metadata, but require more computation. Transfer learning achieved higher test accuracy than the CNN trained from scratch in the saved run. These findings are specific to this dataset and configuration and should be validated before practical deployment.

Conclusion and Future Scope

7.1 Conclusion

This case study explored an AI-powered workflow for fashion product analysis using metadata and images from the Fashion Product Images (Small) dataset. The experiments included perceptron learning, metadata-based category classification, product-year estimation, optimizer comparison, regularization, CNN image classification, and transfer learning using VGG16 and ResNet50.

The saved notebook results demonstrate that the perceptron can provide a useful binary baseline and that transfer-learning models outperform the CNN trained from scratch in the reported image experiment. The study also highlights the importance of validation curves, confusion matrices, and consistent test sets when assessing model performance. These results support further investigation of AI techniques for catalogue assistance, while not constituting evidence of production readiness.

7.2 Limitations

The experiments have several limitations:

- The principal multiclass experiments use the ten most frequent article types rather than the full range of products.
- Image experiments use a capped subset to limit training time and memory requirements.
- Metadata models depend on the completeness and quality of the selected attributes.
- The transfer-learning bases are frozen and are not fine-tuned on the target dataset.
- Results are tied to a specific data split, model configuration, and saved execution environment.

- The study does not include a deployed application, live catalogue integration, or a business-impact evaluation.

7.3 Future Scope

Future work could extend the experiments in the following directions:

- Include more product categories and evaluate class imbalance explicitly.
- Fine-tune pretrained layers and compare them with frozen feature extraction.
- Combine metadata and image embeddings in a multimodal classifier.
- Evaluate precision, recall, F1-score, and per-class performance alongside accuracy.
- Repeat experiments with multiple random seeds and report mean and standard deviation.
- Compare model size, inference latency, memory usage, and accuracy.
- Deploy a selected model through an application programming interface for catalogue review.
- Monitor errors and retrain models as product styles and catalogue data change.

Bibliography

[1] P. Aggarwal, "Fashion Product Images (Small)," Kaggle dataset. Available: https://www.kaggle.com/datasets/paramaggarwal/fashion-product-images-small

[2] TensorFlow, "TensorFlow Documentation." Available: https://www.tensorflow.org/

[3] Keras, "Keras Documentation." Available: https://keras.io/

[4] Scikit-learn, "Scikit-learn User Guide." Available: https://scikit-learn.org/stable/user_guide.html

[5] K. Simonyan and A. Zisserman, "Very Deep Convolutional Networks for Large-Scale Image Recognition," 2014. Available: https://arxiv.org/abs/1409.1556

[6] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," 2015. Available: https://arxiv.org/abs/1512.03385